



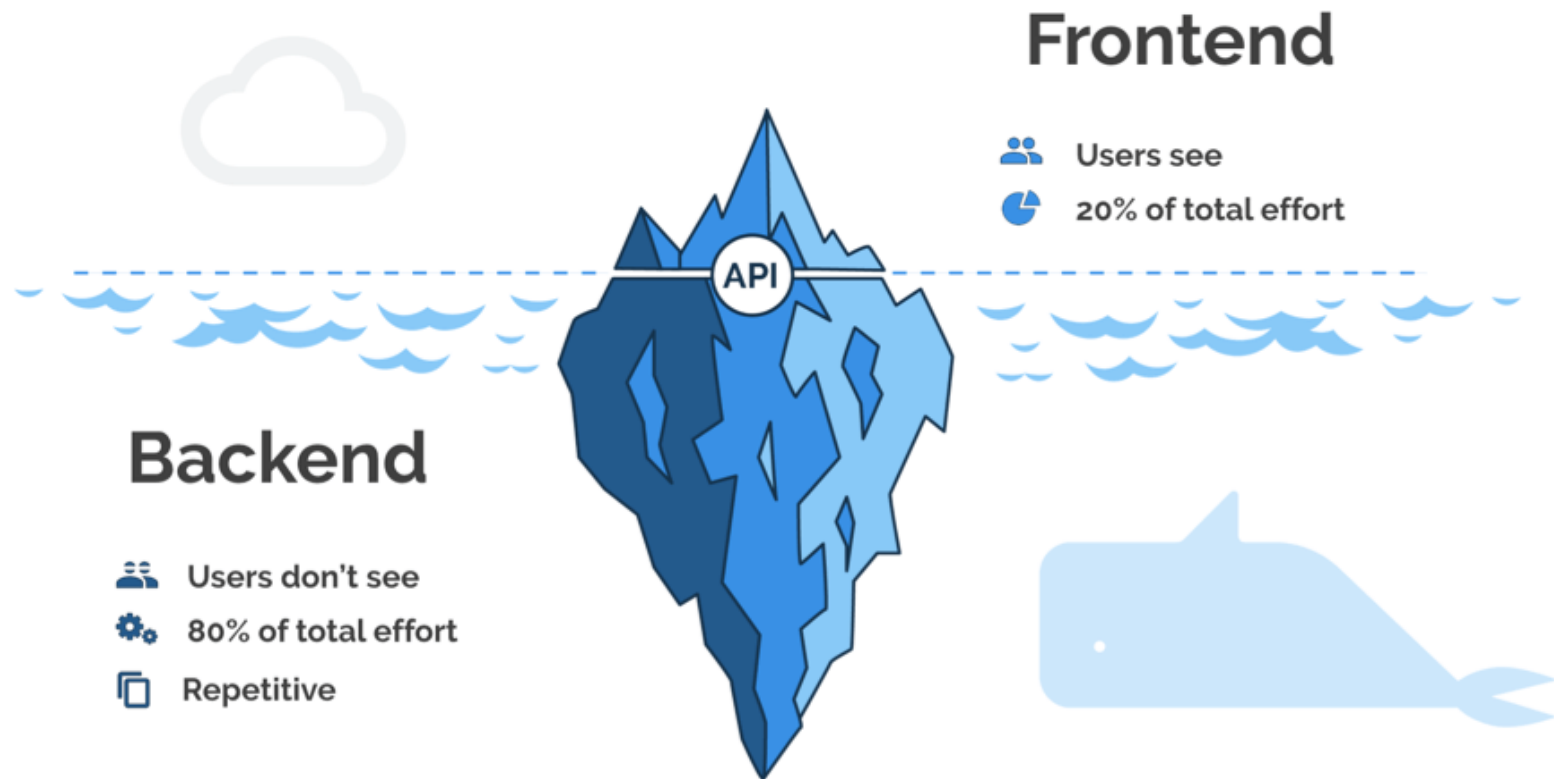
Développement **Mobile**

Programmation et nouvelles technologies

Séance VI

API et Redux

HELLO



API

- **A**pplication **P**rogramming **I**nterface
- Permet d'accéder aux fonctions et aux données d'applications au travers d'une interface applicative
 - Simplicité d'utilisation
 - Harmonisation de la programmation grâce à des conventions
 - Ajoute une couche d'abstraction pour
 - Accéder aux données
 - Modifier les données
 - Supprimer les données
 - Permet d'utiliser plusieurs langages de programmation, voire plusieurs plateformes !
 - Sécuriser l'accès aux données (Autorisation, Authentification)

Exemples d'API

- Api pour accéder aux données d'un service web
- API pour communiquer avec Google Drive, Facebook,...
- API pour accéder aux fonctionnalités de l'OS (fichiers, dossiers,...)
- API du browser (Web Storage, Web Notification...)
- API Java pour discuter avec la JVM

➔ **Des API pour tout type de systèmes et pour toutes applications peu importe le niveau**

Pokemon (type)

Name	Description	Type
id	The identifier for this resource.	integer
name	The name for this resource.	string
base_experience	The base experience gained for defeating this Pokémon.	integer
height	The height of this Pokémon in decimetres.	integer
is_default	Set for exactly one Pokémon used as the default for each species.	boolean
order	Order for sorting. Almost national order, except families are grouped together.	integer
weight	The weight of this Pokémon in hectograms.	integer
abilities	A list of abilities this Pokémon could potentially have.	list PokemonAbility
forms	A list of forms this Pokémon can take on.	list NamedAPIResource (PokemonForm)
game_indices	A list of game indices relevant to Pokémon item by generation.	list VersionGameIndex
held_items	A list of items this Pokémon may be holding when encountered.	list PokemonHeldItem
location_area	A link to a list of location areas, as well as encounter details.	string

REST – Règles d'interface uniforme

- Utiliser les **codes de réponses HTTP** appropriés
 - 200 : Ok (Lors d'un GET)
 - 201 : Created (Lors d'un POST)
 - 204 : No content (Lors d'un DELETE)
 - 401 : utilisateur non authentifié
 - 403 : accès refusé
 - 404 : page non trouvée
 - 500 et 503 : erreur serveur
 - 504 : le serveur n'a pas répondu
 - ...
- 1XX – Information
- 2XX – Succès
- 3XX – Redirection
- 4XX – Erreur du client HTTP
- 5XX – Erreur du serveur

Mise en place avec axios

- Réalise des XMLHttpRequests depuis le navigateur
- Client HTTP basé sur les promesses
- Gère les requêtes et les réponses
- Transforme les données des requêtes et des réponses en JSON

AXIOS

```
npm install --save axios
```

```

import axios from 'axios';
import { useState, useEffect } from 'react';
import { Text, View, ScrollView } from 'react-native';
import { ListItem } from '@rneui/themed';

export default function App() {
  const [data, setData] = useState([])
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    getAPI();
  }, []);

  const getAPI = async () => {
    await axios.get("https://pokeapi.co/api/v2/pokemon?limit=50&offset=0")
    .then((response) => {
      console.warn(response.data.results);
      setData(response.data.results);
    })
    .finally(() => {
      setLoading(false);
    })
  }

  if(loading){
    return(
      <View style={styles.container}>
        <Text>Open up App.js to start working on your ap2paa!</Text>
      </View>
    )
  }else{
    return (
      <ScrollView>
        {data.map((pokemon) => {
          return (
            <ListItem key={pokemon.name} bottomDivider>
              <ListItem.Content>
                <ListItem.Title>{pokemon.name}</ListItem.Title>
              </ListItem.Content>
            </ListItem>
          );
        })}
      </ScrollView >
    );
  }
}

```


SWR

- Bibliothèque React pour la récupération des données
- Permet de gérer les flux de revalidation des données
- Permet de mettre en cache vos données
- [Documentation officielle](#)



SWR

React Hooks for Data Fetching

```
npm install --save swr
```

```
import axios from 'axios';

//Communication avec l'API
export const API = axios.create({
  baseURL: `/api/`,
  withCredentials: true,
  headers: {
    'X-Requested-With' : 'XMLHttpRequest',
    'Accept' : 'application/json',
    'Content-Type' : 'application/json'
  }
});

export const fetcher = url => API.get(url).then(res => res.data);

export default {API,fetcher};
```

```
import React from 'react';
import { View, Text } from 'react-native';
import useSWR from 'swr';

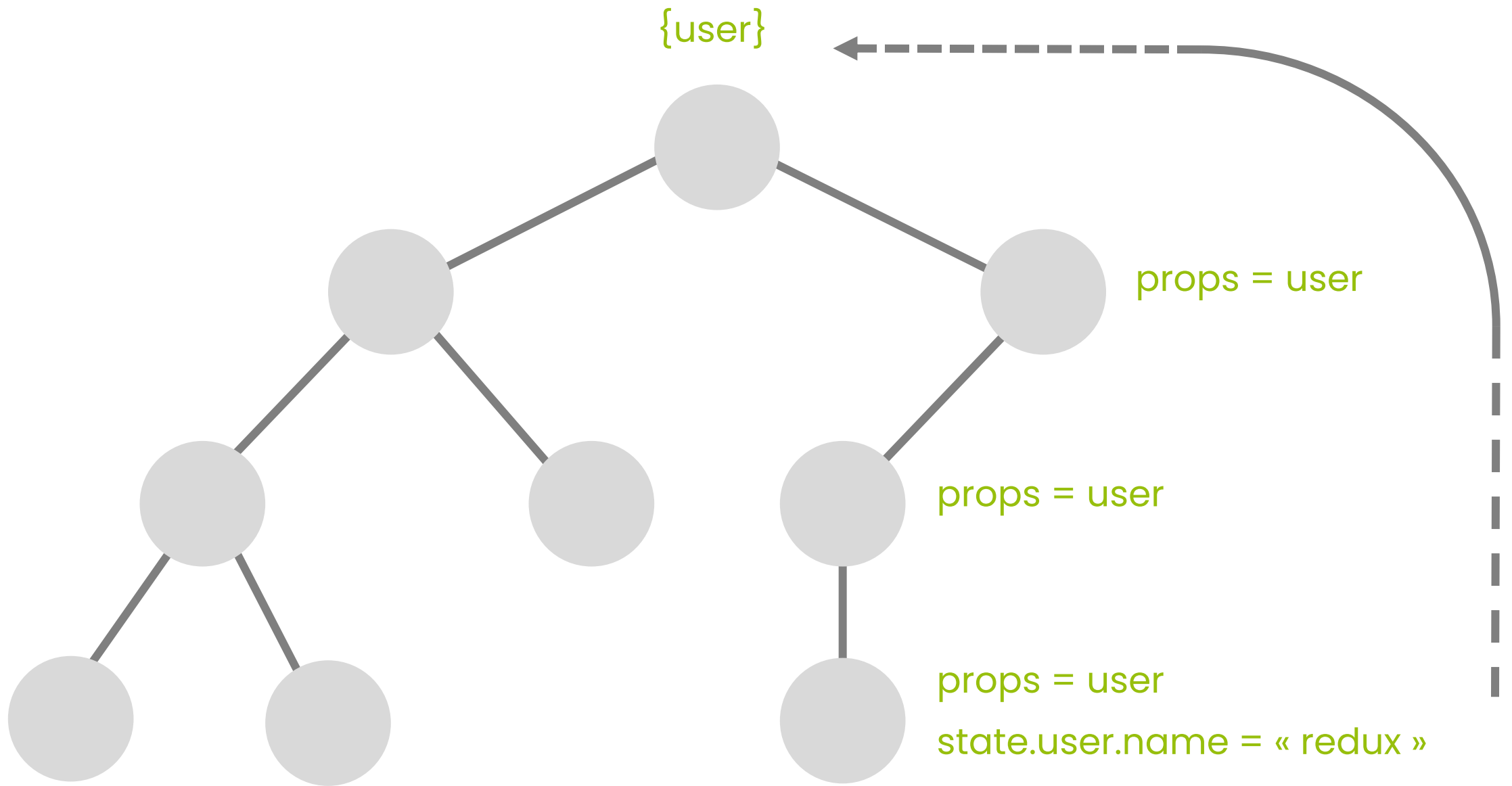
const Profile = () => {
  const { data, error, isLoading } = useSWR('/api/user', fetcher);

  if (error) {
    return (
      <View>
        <Text>Échec du chargement</Text>
      </View>
    );
  }
  if (isLoading) {
    return (
      <View>
        <Text>Chargement...</Text>
      </View>
    );
  }
  return (
    <View>
      <Text>Salut {data.name}!</Text>
    </View>
  );
};

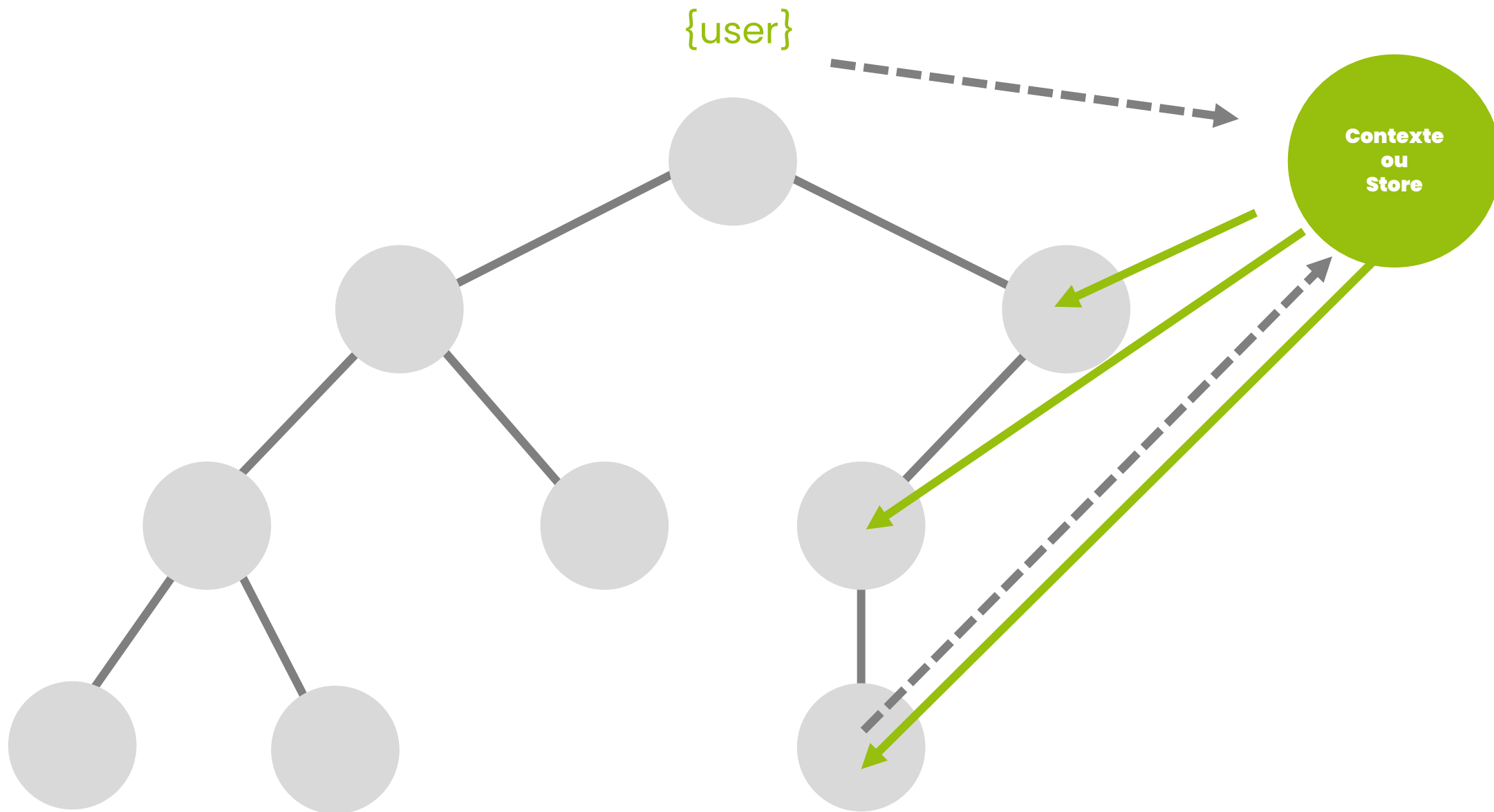
export default Profile;
```

Data Management

HELLO







useContext

- Permet de s'abonner à un contexte depuis un composant
 - Peut recevoir des valeurs
 - Peut recevoir des fonctions
- Nécessité de créer un fournisseur, un Provider du contexte
- Attention à insérer le fournisseur un cran au-dessus du composant qui l'appelle
- Idéal pour gérer un petit ensemble de données, par exemple le thème

Création du
contexte avec
la valeur par
défaut

Création du
state avec la
valeur par
défaut

```
import { createContext, useState } from 'react';  
export const ThemeContext = createContext('blue');  
  
export default function ThemeContextProvider({children}){  
  const [theme, setTheme] = useState( 'blue');  
  
  return(  
    <ThemeContext.Provider value={{theme, setTheme}}>  
      {children}  
    </ThemeContext.Provider>  
  )  
}
```

Fournir
l'ensemble des
valeurs et
fonctions
devant être
utilisées dans
votre code

```
import React from 'react';
import ReactDOM from 'react-dom/client';

import App from './App.jsx';

import ThemeContextProvider from './contexts/themeContext';

ReactDOM.createRoot(document.getElementById('root')).render(
  <ThemeContextProvider>
    <App />
  </ThemeContextProvider>
)
```



Insertion du Provider en
amont du composant qui
va l'utiliser

Exploitation du contexte
ThemeContext, importer les
valeurs et fonctions utiles
pour votre composant



```
import {useContext} from 'react';
import {ThemeContext} from '../contexts/themeContext';

const ThemedText = () => {
  const { theme } = useContext(ThemeContext);

  return (
    <Text>
      Ceci est le thème actuel : {theme}
    </Text>
  );
};
```

Redux

- Framework Open Source Javascript
- Décentralisation de la gestion de vos données dans un store
 - Une seule source de données
 - Complémentaire au framework FrontEnd
- Mécanisme de dispatch des nouvelles données
- Alternative : MobX
- [Documentation officielle](#)



```
npm install @reduxjs/toolkit
npm install react-redux
npm install redux-persist
```



Dan Abramov
@dan_abramov



Following

In case you wondered, the Redux logo is inspired by me watching Charmed when I was ten. charmed.wikia.com/wiki/Triquetra



RETWEETS
24

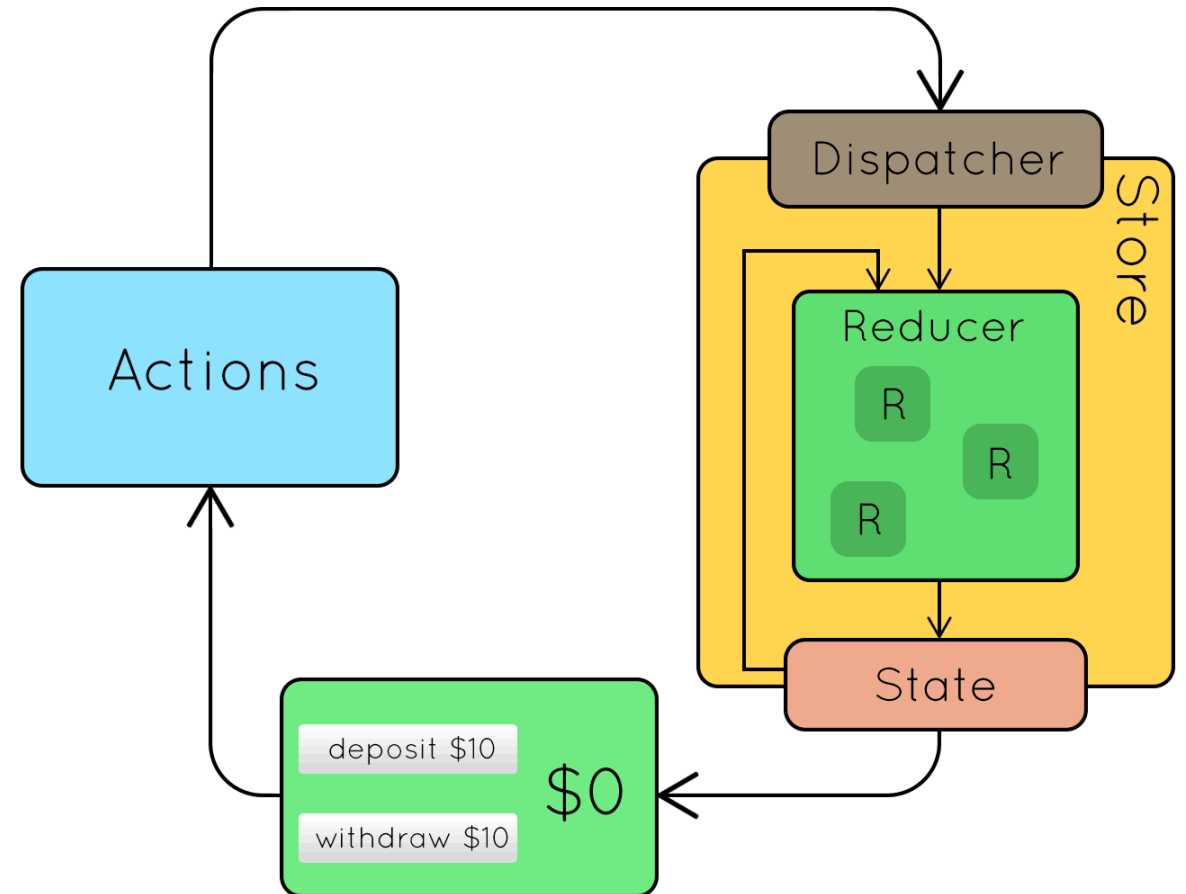
LIKES
94



4:54 PM - 21 May 2016

Étapes de Redux

- Un **store** représente la bibliothèque des données de l'application accessible partout
- Une **action** peut être déclenchée par un **composant** qui va mettre à jour un state
 - Contient un nom d'action et un payload si besoin
- L'action va passer à travers un **dispatcher** pour être redirigé vers le reducer approprié
- Le **reducer** permet de mettre à jour le state au sein du store



Principes

- Le store est la source ultime du savoir
 - Tous vos states doivent être situées au sein du store
 - Possibilité de fonctionner qu'avec une partie de l'information, peut-être moins propre
- Le state est en lecture seule !
 - Toutes les modifications doivent passer par une action CRUD
 - Merci Immer pour gérer l'état des states
- Les changements sont faits dans des fonctions pures
 - La fonction n'altère pas les données entrantes
 - Elle ne dépend pas de state externe
 - Elle donne une même sortie pour une même entrée

Création du store

- Division de votre store en « **slice** »
- Définition de votre **initialState**
- Création de vos **reducers** avec les actions sur votre slice
- **Configuration du Store** qui va contenir l'ensemble des slices
- **PersistStore** pour la conservation des données dans l'AsyncStorage
- Export des **actions**

→ Exemple Pokemon

```
import { configureStore, createSlice } from "@reduxjs/toolkit";

const pokemonsSlice = createSlice({
  name: "pokemons",
  initialState: {
    data: [],
    isLoading: true,
  },
  reducers: {
    setPokemons: (state, action) => {
      state.data = action.payload;
      state.isLoading = false;
    },
  },
})

export const store = configureStore({
  reducer: {
    pokemons: pokemonsSlice.reducer,
  },
})

export const {setPokemons} = pokemonsSlice.actions;
```


Intégration du Provider

- Devient la porte d'entrée entre vos écrans
- Englobe votre application avec le component **Provider** de redux pour lui donner accès au store
- Bien penser à séparer votre application sous la forme d'**écrans**

```
import { Provider } from 'react-redux';
import { store } from '../redux';

import PokemonsScreen from '../screens/PokemonsScreen';

export default function App() {
  return (
    <Provider store={store}>
      <PokemonsScreen/>
    </Provider>
  );
}
```

Dans votre composant

- **useDispatch** pour utiliser vos actions du redux
- **useSelector** pour aller rechercher les states de votre store

```
import axios from 'axios';
import { useEffect } from 'react';
import { Text, View, ScrollView } from 'react-native';
import { ListItem } from '@rneui/themed';

import { useDispatch, useSelector } from 'react-redux';
import { setPokemons } from '../redux';

export default function PokemonsScreen () {
  const pokemons = useSelector((state) => state.pokemons.data)
  const loading = useSelector((state) => state.pokemons.isLoading)

  const dispatch = useDispatch();

  useEffect(() => {
    getAPI();
  }, []);

  const getAPI = async () => {
    await axios.get("https://pokeapi.co/api/v2/pokemon?limit=50&offset=0")
      .then((response) => {
        dispatch(setPokemons(response.data.results));
      })
  }

  if(loading){
    return(
      <View>
        <Text>Open up App.js to start working on your ap2paa!</Text>
      </View>
    )
  }
  else{
    return(
      <ScrollView>
        {pokemons.map((pokemon, index) => {
          return (
            <ListItem key={index} bottomDivider>
              <ListItem.Content>
                <ListItem.Title>{pokemon.name}</ListItem.Title>
                </ListItem.Content>
              </ListItem>
            );
          )}}
        </ScrollView >
    )
  }
}
```

Stockage des données

HELLO

AsyncStorage

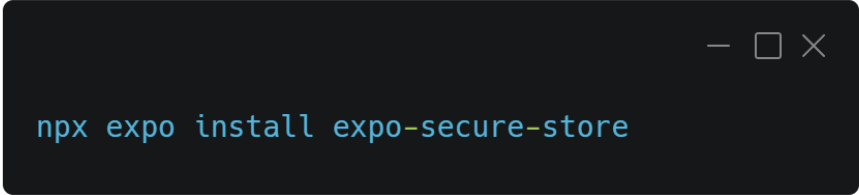
- AsyncStorage est une API native fournie par React Native qui permet de stocker des données localement sous forme de paires clé-valeur.
- **Capacité limitée**, utilisation pour des données de petites tailles
- Écriture et lecture des données de manière asynchrone
- Non sécurisé
- Disponible sous Expo
- Utilisée par redux-persist



```
npm install @react-native-async-storage/async-storage
```

Secure Storage

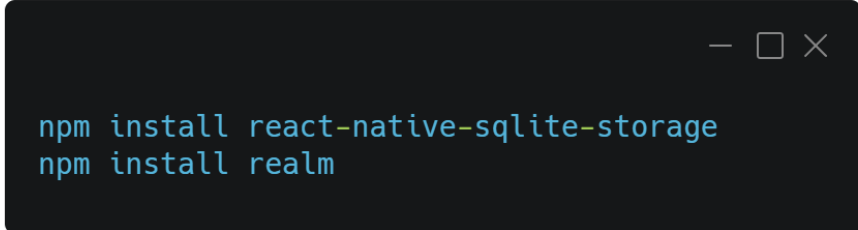
- Stockage sécurisé pour des données sensibles, idéal pour stocker des tokens JWT
- Cryptage natif pour chiffrer les données localement
- Plus lent que l'AsyncStorage, garder son utilisation limitée en quantité de données
- Disponible sous Expo



```
npx expo install expo-secure-store
```

Bases de données locales

- SQLite ou Realm disponibles sous mobile
- Permet une gestion de grandes quantités de données structurées
- Permet d'effectuer des requêtes de type SQL
- Idéal pour la persistance des données pour un mode total hors-ligne
 - Attention à la resynchronisation des données lors de la récupération d'internet

A dark-themed terminal window with a title bar containing a minus sign, a square icon, and a close 'X' icon. The terminal displays two lines of text in a light blue font: 'npm install react-native-sqlite-storage' and 'npm install realm'.

```
npm install react-native-sqlite-storage
npm install realm
```

```

import React, { useState, useEffect } from 'react';
import { View, Button, Text } from 'react-native';
import * as SecureStore from 'expo-secure-store';

export default function App() {
  const [token, setToken] = useState(null);

  const saveToken = async (jwt) => {
    await SecureStore.setItemAsync('userToken', jwt);
    setToken(jwt);
  };

  const loadToken = async () => {
    const storedToken = await SecureStore.getItemAsync('userToken');
    setToken(storedToken || null);
  };

  const deleteToken = async () => {
    await SecureStore.deleteItemAsync('userToken');
    setToken(null);
  };

  useEffect(() => {
    loadToken();
  }, []);

  return (
    <View>
      <Text>{token}</Text>
    </View>
  );
}

```